

Universidade de São Paulo

Instituto de Matemática, Estatística e Ciência da Computação

OBI 2025 - Fase 2 - Nível Sênior

Editorial

Autores: Gabriela Kwong Yamaguti

Explicações dos Problemas

Arara!

Tags: Matemática

Como queremos colocar N araras com no mínimo 4 gaiolas entre as araras, a forma mais compacta de organizar as aves é colocar a primeira arara na primeira gaiola e, a partir dela, deixar 4 gaiolas vazias antes de colocar uma próxima arara.

Podemos fazer a contagem da seguinte maneira: precisamos de N gaiolas para as N aves, e depois precisamos de 4 gaiolas para cada um dos $N - 1$ espaços entre as N aves. Desse modo, o espaço mínimo total necessário na linha será de $N + (4 \times (N - 1))$.

O problema se resume, portanto, a checar se a quantidade total de gaiolas disponíveis (M) é maior ou igual a esse valor mínimo.

Mania de Ímpar

Tags: Matriz

A regra principal é que a soma de dois cookies vizinhos deve ser sempre ímpar. Para que a soma de dois números seja ímpar, obrigatoriamente um deles deve ser par e o outro deve ser ímpar. Isso significa que a nossa matriz de cookies precisa alternar posições pares e ímpares.

Portanto, existem apenas duas configurações finais válidas para a bandeja: ou o primeiro cookie da posição $(0, 0)$ vira ímpar, ou ele vira par. (A paridade desse cookie irá definir a paridade do resto dos cookies) .

Para resolver o problema gastando o mínimo de gotas, a estratégia ideal é testar essas duas únicas possibilidades e ver qual delas exige menos alterações. Como adicionar uma gota de chocolate muda a paridade de um número (transforma par em ímpar e vice-versa), o custo para consertar um cookie errado será sempre de exatamente 1 gota.

Sabendo disso, podemos usar a soma dos índices da linha e da coluna $(i + j)$ para mapear as casas do "tabuleiro". Se a paridade atual do cookie não bater com o padrão esperado daquela posição, precisaremos somar 1 a ele .

No código, primeiro lemos a matriz e calculamos quantas gotas seriam gastas no primeiro cenário (onde a posição $(0, 0)$ é ímpar), guardando o resultado em `resp1`. Como

o total de cookies na bandeja é $N \times M$ e cada cookie invertido em um padrão estaria correto no outro, o custo do segundo cenário (**resp2**) é simplesmente o total de cookies menos **resp1**.

Descobrimos o melhor caso comparando os dois valores e, por fim, passamos alterando a matriz original e imprimindo o resultado: se o cookie precisava mudar de paridade, somamos 1 a ele; caso contrário, mantemos seu valor original.

Um desafio muito distinto

Tags: Busca Binária

O objetivo de João é fazer o maior número possível de rodadas sem que o total de bolinhas na caixa atinja ou ultrapasse L antes da última jogada permitida. Como ele quer maximizar as rodadas e não pode repetir valores de bolinhas já escolhidos, a melhor estratégia é sempre escolher os menores valores disponíveis, ou seja, começar colocando A bolinhas, depois $A + 1$, e assim por diante.

Se ele usar todas as opções possíveis de A até B e a soma ainda for menor ou igual a L , o total de rodadas será simplesmente a quantidade de números nesse intervalo. Caso a soma total de todas as opções ultrapasse L , o jogo vai acabar mais cedo.

Para descobrir exatamente em qual rodada o jogo termina, precisamos encontrar o menor valor limite M (onde $A \leq M \leq B$) que faz a soma acumulada atingir ou passar de L . Para calcular a soma de A até qualquer número de forma rápida, usamos a fórmula da soma da PA:

$$\text{Soma} = \frac{(\text{quantidade de termos}) \times (\text{primeiro termo} + \text{último termo})}{2}$$

Como os limites do problema são gigantescos (L pode chegar a 10^{18}), testar valor por valor somando um a um seria lento demais. A estratégia ideal para encontrar esse ponto de parada rapidamente é aplicar uma busca binária no intervalo entre A e B .

No código, para cada partida, primeiro checamos se a soma de todo o intervalo de A até B é menor ou igual a L . Se for, João consegue usar todos os valores e a resposta é $b - a + 1$. Caso contrário, iniciamos a busca binária para achar o menor valor M que estoura o limite L . O número de rodadas completadas com sucesso até o jogo acabar será a quantidade de números de A até esse ponto M , calculada como $\text{resp} - a + 1$.

Feira de Artesanato

Tags: STL

Precisamos gerenciar o estoque de duas formas diferentes: Quando entra um cliente decidido, precisamos encontrar rapidamente o objeto mais barato de um tipo específico. Já quando entra um cliente indeciso, precisamos encontrar o objeto mais barato de toda a loja (usando o menor número de tipo como desempate).

Como os produtos comprados deixam de existir, a estratégia ideal é manter duas representações do estoque que se atualizam juntas: um grande conjunto ordenado para a loja inteira e uma lista de conjuntos separados para cada tipo de objeto. Para simular isso no código de forma eficiente, usamos a estrutura `std::set`, que mantém os elementos sempre ordenados automaticamente.

Criamos o conjunto `geral`, que organiza os produtos por preço e usa o tipo como critério de desempate, e um vetor de conjuntos chamado `estoque`, onde cada posição

guarda os produtos organizados por preço dentro de seu próprio tipo.

Quando um cliente indeciso chega ($u == 0$), pegamos o primeiro elemento do conjunto geral, somamos seu preço na resposta e o removemos de ambos os lugares. Quando chega um cliente decidido ($u > 0$), fazemos o mesmo processo, mas buscando diretamente no conjunto do tipo u . É importante lembrar que a estrutura `set` não guarda mais de um elemento igual, por isso devemos usar um `id`, para diferenciar quando temos objetos com o mesmo preço.

Implementações

Arara

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int32_t main()
6 {
7     int n, m;
8     cin >> n >> m;
9     if (m >= n + (n - 1) * 4)
10         cout << "S";
11     else
12         cout << "N";
13 }
```

Mania de Ímpar

```
1
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int32_t main()
6 {
7     int n, m;
8     cin >> n >> m;
9     int mat[n][m];
10     int resp1 = 0, resp2 = 0; // caso (0,0) seja impar ou par respectivamente
11     for (int i = 0; i < n; i++)
12     {
13         for (int j = 0; j < m; j++)
14         {
15             cin >> mat[i][j];
16         }
17     }
18     // caso em que (0,0) sera impar
19     for (int i = 0; i < n; i++)
20     {
21         for (int j = 0; j < m; j++)
22         {
23             if (mat[i][j] % 2 == (i + j) % 2)
24                 resp1++;
25         }
26     }
27     // caso que (0,0) é par
```

```

28     resp2 = (m * n) - resp1;
29     int melhor = (resp1 <= resp2) ? 0 : 1;
30     cout << min(resp1, resp2) << endl;
31     for (int i = 0; i < n; i++)
32     {
33         for (int j = 0; j < m; j++)
34         {
35             if (!melhor)
36                 mat[i][j] += mat[i][j] % 2 == (i + j) % 2;
37             else
38                 mat[i][j] += mat[i][j] % 2 != (i + j) % 2;
39             cout << mat[i][j] << " ";
40         }
41         cout << endl;
42     }
43 }

```

Um desafio muito distinto

```

1
2 #include <bits/stdc++.h>
3 using namespace std;
4 #define int long long
5 int32_t main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(NULL);
9     int p;
10    cin >> p;
11    while (p--)
12    {
13        int l, a, b;
14        cin >> l >> a >> b;
15        if ((b - a + 1) * (b + a) / 2 <= l)
16            cout << b - a + 1 << endl;
17        else
18        {
19            int e = a, d = b;
20            int resp = -1;
21            while (e <= d)
22            {
23                int m = e + (d - e) / 2;
24                if ((m - a + 1) * (m + a) / 2 >= l)
25                {
26                    resp = m;
27                    d = m - 1;
28                }
29                else

```

```

30         e = m + 1;
31     }
32     cout << resp - a + 1 << endl;
33 }
34 }
35 }

```

Feira de Artesanato

```

1
2  #include <bits/stdc++.h>
3  using namespace std;
4  #define int long long
5  int32_t main()
6  {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9      int n, m;
10     cin >> n >> m;
11     vector<int> tipo(n);
12     vector<int> preco(n);
13     map<int, vector<int>> aux;
14     for (int i = 0; i < n; i++)
15         cin >> tipo[i];
16     for (int i = 0; i < n; i++)
17         cin >> preco[i];
18     // preco, tipo , id
19     set<pair<int, pair<int, int>>> geral;
20     // preco, id
21     vector<set<pair<int, int>>> estoque(m + 1);
22     for (int i = 0; i < n; i++)
23     {
24         geral.insert({preco[i], {tipo[i], i}});
25         estoque[tipo[i]].insert({preco[i], i});
26     }
27     int c;
28     cin >> c;
29     int resp = 0;
30     for (int i = 0; i < c; i++)
31     {
32         int u;
33         cin >> u;
34         if (!u)
35         {
36             if (!geral.empty())
37             {
38                 auto [p, aux] = *geral.begin();
39                 resp += p;

```

```
40         geral.erase(geral.begin());
41         estoque[aux.first].erase({p, aux.second});
42     }
43 }
44 else
45 {
46     if (!estoque[u].empty())
47     {
48         auto [p, id] = *estoque[u].begin();
49         resp += p;
50         estoque[u].erase(estoque[u].begin());
51         geral.erase({p, {u, id}});
52     }
53 }
54 }
55 cout << resp;
56 }
```
